bge-m3

1、原理介绍

（1）bge的基本步骤包括哪些：

主要步骤包括切块、向量化、检索、生成。

（2）你当时做bge用的哪个模型，目前的bge模型有哪些。

我当时使用的是bge-v1.5-large-zh模型，专门为中文检索而适配的模型。

那么现在有哪些bge模型呢,具体如下：

系列	代表模型	和 `bge-large-v1.5` 的主要区别	适合场景
BGE v1.5 dense embedding	`bge-large/base/small-en-v1.5`、`bge-large/base/small-zh-v1.5`	传统 dense 向量模型；large 通常 1024 维，base/small 更轻；最大长度多为 512。(huggingface.co)	单语 RAG、语义搜索、成本敏感场景
BGE-M3	`BAAI/bge-m3`	多语言、最长 8192 tokens，同时支持 dense、sparse lexical、multi-vector/ColBERT 三种检索方式；dense 维度 1024。(huggingface.co)	现在做通用 RAG 的首选之一，尤其是中英混合、多语言、长文档、混合检索
LLM-based multilingual embedding	`BAAI/bge-multilingual-gemma2`	基于 Gemma-2-9B 的多语言 embedding，参数量 9B；官方示例使用 4096 max length，面向多语言与多任务。(huggingface.co)	追求高质量、多语言，且能接受显著更高推理成本
ICL embedding	`BAAI/bge-en-icl`	7B 英文 embedding，支持 few-shot/in-context learning；可在 query 里给任务样例提升新任务效果。(huggingface.co)	英文复杂检索、领域任务少样本适配
代码 embedding	`BAAI/bge-code-v1`	2B，面向代码检索、文本检索、多语言检索；支持英文/中文自然语言 query 和 20 种编程语言；官方示例 max length 4096。(huggingface.co)	代码库 RAG、Text-to-SQL、代码相似、bug/solution 检索

Reasoner embedding	`BAAI/bge-reasoner-embed-qwen3-8b-0923`	基于 Qwen3-8B，专门面向 reasoning-intensive retrieval；官方称在 BRIGHT 上 original query nDCG@10 = 37.1。(huggingface.co)	数学、定理、复杂推理型问答、需要"理解推理链"的检索
多模态 BGE-VL	`BGE-VL-base/large`、`BGE-VL-Screenshot`、`BGE-VL-v1.5-*`	支持文本、图片、图文组合、截图等多模态检索；如 `BGE-VL-large` 输出 768 维，`BGE-VL-base` 输出 512 维，`BGE-VL-Screenshot` 输出 2048 维。(huggingface.co)	图像搜索、截图检索、图文混合知识库
Reranker 系列	`bge-reranker-v2-m3`、`v2-gemma`、`v2-minicpm-layerwise`、`v2.5-gemma2-lightweight`、`base/large`	不是 embedding，不产向量；输入 query+passage 后直接输出相关性分数。官方建议用于 embedding 初召后的 top-k 重排。(huggingface.co)	提升最终排序质量；RAG 检索 top 50/100 后重排

bge-m3

bge-m3不仅仅是传统的向量，而且还有sparse lexical、multi-vector等多个功能。

dense

输入一句话，将其改为向量化编码，如1024维向量。

sparse lexical

保留"关键词权重"的稀疏表示 Sparse lexical 可以理解成更聪明的关键词检索，类似 BM25，但由模型学习词的重要性。

它不是把一句话压成一个稠密向量，而是把文本表示成很多词的权重，例如：

```
{
  "苹果": 0.8,
  "手机": 0.6,
  "电池": 1.2,
  "发热": 1.5,
  "怎么办": 0.3
}
```

优点

- 对关键词、型号、术语、错误码、专有名词特别友好。
- 可解释性更强，因为你能看到哪些词贡献了匹配。
- 适合技术文档、API 文档、日志、代码、论文、产品手册。

缺点

- 语义泛化能力弱于 dense。
- 如果用户和文档用词完全不同，它可能找不到。
- 比如用户搜“手机很烫”，文档写“设备温度异常升高”，sparse 可能不如 dense。

Multi-vector

一个文本变成多个向量，做细粒度匹配

普通 dense 是：

text

```
query -> 1 个向量
doc   -> 1 个向量
```

multi-vector

text

```
query -> 多个向量
doc   -> 多个向量
```

比如 query：

> 苹果手机电池发热怎么办?

它可能给每个重要 token 或片段生成向量：

text

```
苹果手机 -> 向量 A
电池     -> 向量 B
发热     -> 向量 C
怎么办   -> 向量 D
```

文档也会有很多向量：

text

```
iPhone      -> 向量 1
battery     -> 向量 2
overheating -> 向量 3
solution    -> 向量 4
...
```

检索时不是简单比较“query 整体”和“doc 整体”，而是看 query 的每个局部是否都能在 doc 里找到对应内容。

这类方法常见代表是 ColBERT 风格的 late interaction。

优点

- 匹配更细，效果通常比单向量 dense 更好。
- 对长文档、多主题文档更友好。

- 能减少“整篇看起来相关，但关键点没覆盖”的情况。
- 适合高质量检索、rerank、复杂 query。

缺点

- 存储成本高，因为一个文档不止一个向量。
- 检索计算更贵。
- 工程复杂度高，不是所有向量数据库都原生支持。

新项目下，这些技术要怎么使用：

最简单只使用dense:

```
query -> dense vector -> 向量库检索 -> top-k 文档
```

更稳的方案

dense + sparse lexical

```
dense 负责语义召回
sparse 负责关键词召回
最后把分数融合
```

适合技术文档、产品文档、代码文档、知识库问答。

更高质量的方案：

```
dense + sparse lexical 做初召
multi-vector 或 reranker 做精排
```

dense 看意思，sparse 看关键词，multi-vector 看局部细节。 最常见的生产组合是：**dense + sparse 做召回，reranker 或 multi-vector 做精排。**

ColBERT 模式

指 ColBERT（Contextualized Late Interaction over BERT） 式检索：不做"一句话一个向量"，而是保留每个 token 的向量。打分时查询 token 和文档 token 两两算相似度，每个查询 token 取最大相似度再求和。所以它比 dense 更细粒度，能捕捉局部匹配。脚本里通过 return_colbert_vecs=True 拿到逐 token 向量，再用 compute_colbert_score 算分。

具体的操作如下：

假设 query 有 3 个 token，文档有 4 个 token。

query token 向量：

$$Q = [q_1, q_2, q_3]$$

doc token 向量：

$$D = [d_1, d_2, d_3, d_4]$$

先算两两点积矩阵：

$$M = \begin{bmatrix} q_1 \cdot d_1 & q_1 \cdot d_2 & q_1 \cdot d_3 & q_1 \cdot d_4 \\ q_2 \cdot d_1 & q_2 \cdot d_2 & q_2 \cdot d_3 & q_2 \cdot d_4 \\ q_3 \cdot d_1 & q_3 \cdot d_2 & q_3 \cdot d_3 & q_3 \cdot d_4 \end{bmatrix}$$

然后每一行取最大值：

$$[\max M_{1,:}, \max M_{2,:}, \max M_{3,:}]$$

再求和：

$$\text{score}(q, d) = \max M_{1,:} + \max M_{2,:} + \max M_{3,:}$$

Hybrid查询

Hybrid 分数 = (1.0×dense分 + 0.3×sparse分 + 1.0×colbert分) / (1.0+0.3+1.0) 三种分数加权平均的结果。

2、结果查询

dense 结果

```
Q: 中兴通讯的5G技术怎么样?
  1. [d0] 0.7615  中兴通讯是全球领先的综合通信解决方案提供商，在5G基站、核心网和光传输领域拥有强大实力。
  2. [d30] 0.7495  中兴 5G 属于全球第一梯队，基站、核心网端到端能力成熟，Massive MIMO、节能、行业专网、
  3. [d1] 0.6126  5G网络相比4G具有更高的传输速率、更低的时延和更大的连接密度，可支持万物互联。

Q: 什么是机器学习?
  1. [d2] 0.7724  机器学习是人工智能的核心分支，通过数据训练模型来实现预测和分类任务。
  2. [d3] 0.5958  深度学习利用多层神经网络自动提取数据特征，在图像识别和语音识别领域取得突破。
  3. [d21] 0.5734  Machine learning algorithms learn patterns from da
```

从结果来看，dense的查询是最好的，几乎没有偏差。

sparse 结果

```
Q: 中兴通讯的5G技术怎么样?
  1. [d0] 0.1920  中兴通讯是全球领先的综合通信解决方案提供商，在5G基站、核心网和光传输领域拥有强大实力。
  2. [d30] 0.1262  中兴 5G 属于全球第一梯队，基站、核心网端到端能力成熟，Massive MIMO、节能、行业专网、
  3. [d17] 0.0484  数据加密技术通过算法将明文转换为密文，保障信息传输和存储的机密性。

Q: 什么是机器学习?
  1. [d2] 0.2195  机器学习是人工智能的核心分支，通过数据训练模型来实现预测和分类任务。
  2. [d3] 0.1234  深度学习利用多层神经网络自动提取数据特征，在图像识别和语音识别领域取得突破。
  3. [d4] 0.0538  自然语言处理（NLP）研究计算机与人类语言之间的交互，包括机器翻译、问答系统和情感分析。
```

sparse的结果中 q1的查询d17和q2的查询d4的结果都不佳。

hybrid 结果

```
Q: 中兴通讯的5G技术怎么样?
  1. [d0] 0.7204  中兴通讯是全球领先的综合通信解决方案提供商，在5G基站、核心网和光传输领域拥有强大实力。
  2. [d30] 0.6917  中兴 5G 属于全球第一梯队，基站、核心网端到端能力成熟，Massive MIMO、节能、行业专网、
  3. [d1] 0.5579  5G网络相比4G具有更高的传输速率、更低的时延和更大的连接密度，可支持万物互联。

Q: 什么是机器学习?
  1. [d2] 0.7942  机器学习是人工智能的核心分支，通过数据训练模型来实现预测和分类任务。
  2. [d3] 0.6181  深度学习利用多层神经网络自动提取数据特征，在图像识别和语音识别领域取得突破。
  3. [d21] 0.5581  Machine learning algorithms learn patterns from da
```

hybrid的查询结果明显比sparse的结果要好，基本和dense 结果相当。

colbert 结果

```
Q: 中兴通讯的5G技术怎么样?
  1. [d0] 0.8379  中兴通讯是全球领先的综合通信解决方案提供商，在5G基站、核心网和光传输领域拥有强大实力。
  2. [d30] 0.8034  中兴 5G 属于全球第一梯队，基站、核心网端到端能力成熟，Massive MIMO、节能、行业专网、
  3. [d1] 0.6563  5G网络相比4G具有更高的传输速率、更低的时延和更大的连接密度，可支持万物互联。

Q: 什么是机器学习?
  1. [d2] 0.9884  机器学习是人工智能的核心分支，通过数据训练模型来实现预测和分类任务。
  2. [d3] 0.7889  深度学习利用多层神经网络自动提取数据特征，在图像识别和语音识别领域取得突破。
  3. [d21] 0.7103  Machine learning algorithms learn patterns from da
```

colbert的结果也比较好和dense结果一致。事实证明向量查询基本和colbert、hybrid的结果一致。sparse 查询效果稍微差一点。

这里分析下原因吧，为什么sparse 的结果差呢：

```
查询 [q0] '中兴通讯的5G技术怎么样? ' 的 top-12 词权重:
  '兴'                    weight=0.229
  '技术'                   weight=0.192
  '5'                    weight=0.189
  'G'                    weight=0.179
  '通讯'                   weight=0.146
  '怎么样'                  weight=0.139
  '中'                    weight=0.056
  '?'                    weight=0.044
  '的'                    weight=0.013

=== 查询 [q0] '中兴通讯的5G技术怎么样? ' 对各文档打分 ===

[d0] 中兴通讯是全球领先的综合通信解决方案提供商，在5G基站、核心网和光传输领域拥有强大实力。...
sparse=0.192   dense=0.762
共同token贡献: '兴'(0.23x0.36=0.083)  '通讯'(0.15x0.29=0.042)  '5'(0.19x0.16=0.030)
'G'(0.18x0.14=0.026)  '中'(0.06x0.18=0.010)  '的'(0.01x0.06=0.001)

[d20] ZTE Corporation is a leading global provider of ...
sparse=0.034   dense=0.524
共同token贡献: 'G'(0.18x0.19=0.034)

[d30] 中兴 5G 属于全球第一梯队，基站、核心网端到端能力成熟，Massive MIMO、节能、行业专...
sparse=0.126   dense=0.750
共同token贡献: '兴'(0.23x0.31=0.070)  'G'(0.18x0.27=0.048)  '中'(0.06x0.15=0.008)

[d31] 中兴  zte chisi...
sparse=0.045   dense=0.569
共同token贡献: '兴'(0.23x0.19=0.044)  '中'(0.06x0.03=0.001)
```

至于为什么中兴通讯的5G技术怎么样会查询到，原因是“技术”和“的”的权重太高了，会导致整体的检索效果很差。

链路是这样的：

```
1. 查询里"技术"权重 0.19（"5G技术"是查询的重点词之一，模型学出来的权重不低）
2. d17 里"技术"权重高达 0.25（"数据加密技术"同样是 d17 的重点词）
3. sparse 打分是共同 token 权重乘积求和，两边都是高权词 → 光这一项就 0.047
4. 而真正相关的 d20（英文）只能靠一个 "G" 匹配 0.034——所以 d17 在 sparse 里反而排到 d20 前面
```

3、相关问题

1、bge-m3和普通的bge有什么不同

bge-m3共有三种词语或者句子的表现方式，分别是dense、sparse lexical、multi vector三种形式，相比于普通的向量检索模型，sparse lexical和multi vector是另外两种，分别是关键词权重匹配和，关键词向量匹配，一个是给句子中不同的词语给与相应的权重，另一个是给句子中不同的词语给与不同的词向量。分别进行匹配和检索

2、multi vector中的词向量是怎么来的

bge-m3的主要训练原理还是robert-transformer模型的改版，所以基本还是遵循bert的向量编码原理，输入句子，对句子进行分词，对每个分词的结果得到对应的词向量。其实还是一个embedding的结果

3、multi vector中的词向量在不同的句子中，向量的格式一样吗

格式不一样，不同的词在不同的句子中是不同的向量表示，原因如下，bge-m3整体来看仍然是一个transformer模型，那么在计算每个词语的词向量时必然会进行transformer的self-attention操作,也就是说一个词语会和整个句子的其他词语做一个向量与权重的融合计算，而每个句子的词语不同，同一个词语得到的词向量也不同。

4、从你检索的结果来看，为什么sparse的结果。

从结果来看，为什么sparse的检索效果不好，因为sparse是通过字和字的相似度比对进行检索的，如果两个句子中一样的字很多，且权重较大的话，那么很可能在空间层面被认定为相似。比如高兴和中兴两个兴，在一定程度上就会进行相似度的计算。所以句子中的相似字或者词很多的时候有可能就会被认为两者相似。

5、bge的相似度检索和reranker之间有什么区别

检索和reranker都是搜索，但是检索是粗排，相当于在几百万的文档中找出跟用户输入的query相关的句子，reranker是精排，相当于在检索的基础找出跟问题更相关的topk。

那么两者的区别在哪呢：

1、检索是分别向量化，即输入q和文档d，

在 BGE 体系里，常见做法是用 bge-large-zh、bge-m3 这类 embedding 模型：

（1）把用户问题编码成一个向量 （2）把知识库里的每段文本也编码成向量 （3）用余弦相似度 / 内积等方法找最接近的文本

主要特点是：

（1）**速度快**

（2）**适合大规模检索**

（3）文档向量可以**提前离线计算**

（4）可以配合 FAISS、Milvus、ES 等向量库

缺点：query和document交互不充分，大致相关不一定完全符合query的意图

（2）一般来说检索的排序精度不如reranker

2、reranker

精排，主要的作用是对检索后的内容进一步的排序。具体的做法为：

BGE 的 reranker 一般是 **cross-encoder** 思路，例如把：

- query：`bge的相似度检索和reranker区别`
- passage：`BGE reranker 用于对召回结果进行重排序...`

一起输入模型，让模型直接输出一个相关性分数。

特点：

query 和文档一起看

能做更细粒度的相关性判断

常用于对 top-k 候选结果重排

缺点：reranker比较慢，不能对所有的文档都进行一次reranker,reranker是联合建模，设问题为q，待排序文档为d,则reranker的做法为score(q,d)得到相关性的分数。

所以，reranker一般是放在检索后面使用的，**embedding 检索负责从 100 万条里找出 100 条**

reranker 负责从这 100 条里找出最好的 5 条。